Q1)

a) A*

Iterations	Expanded node	Frontier	Path cost g(n)	f-value f(n) = g(n) + h(n)
0	-	S	-	-
1	S	B,A,C	0	0+8 = 8
2	B	A,F,D,C,G3	1	1+1 = 2
3	A	F,D,C,G1,G3	3	3+2 = 5
4	F	D,C,G1,G3	3	3+3 = 6
5	D	E,G2,C,G1,G3	4	4+4 = 8
6	E	G1,G2,C,G3	6	6+1 = 7
7	G1	G2,C,G3	8	8+0 = 8

Path : S → B → F → D → E → G1
Total path cost : g(G1) = 8

b) Uniform cost search

Iterations	Expanded node	Frontier	Path cost g(n)
0	-	S	-
1	S	B,A,C	0
2	B	A,F,D,C,G3	1
3	A	F,C,D,G1,G3	3
4	F	D,C,G1,G3	3
5	D	C,E,G2,G1,G3	4
6	C	E,G2,G1,G3	5
7	E	G1,G2,G3	6
8	G1	G2,G3	8

Path : S → B → F → D → E → G1
Total path cost : g(G1) = 8

c) Iterative deepening A*
Threshold = h(S) = 8

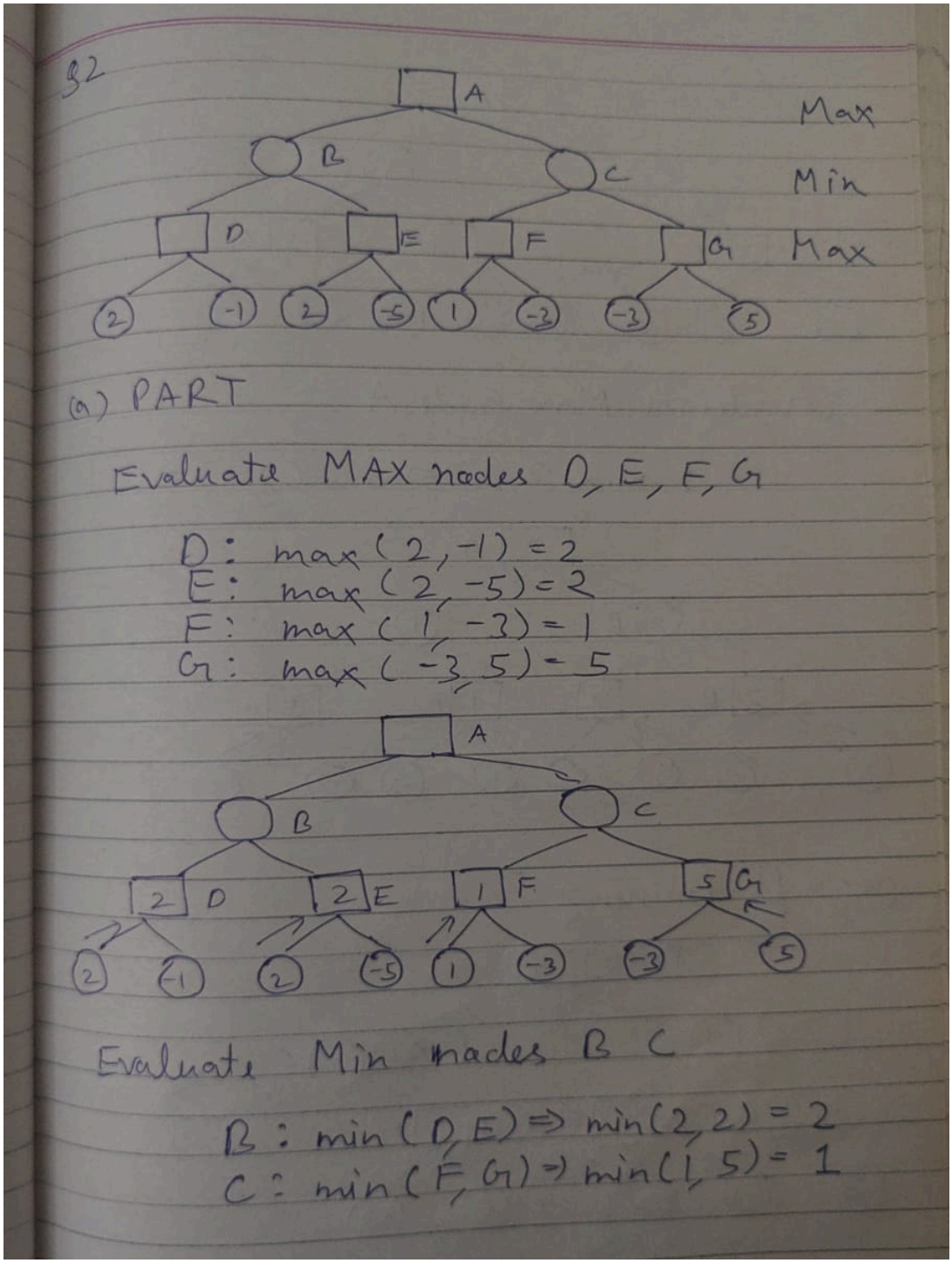
Iterations	Expanded node	Frontier	Reached Nodes	Path cost g(n)
0	-	S	S	-
1	S	A,B	S,A,B,C	0
2	A	B,D,G1	S,A,B,C,D,G1	3

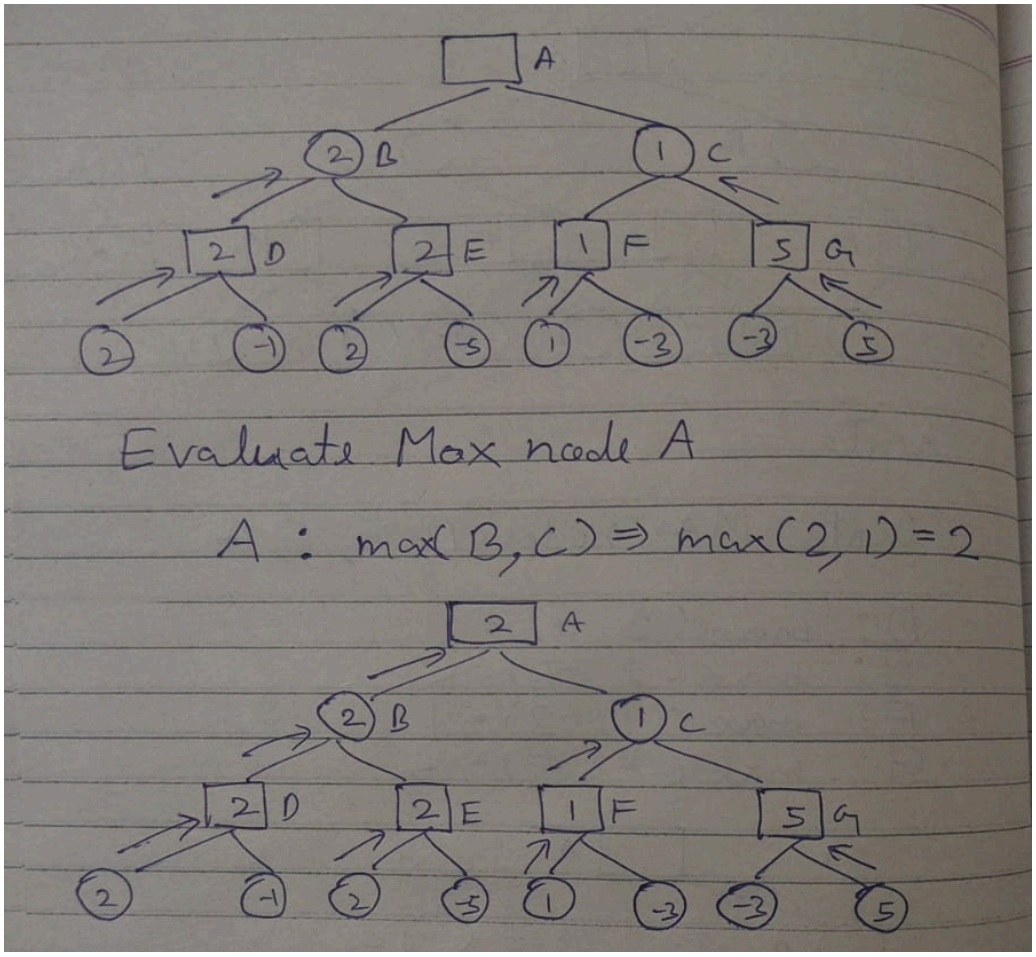
Path : S → A → G1
Total path cost : g(G1) = 3 + 13 = 16

Q2) - reference

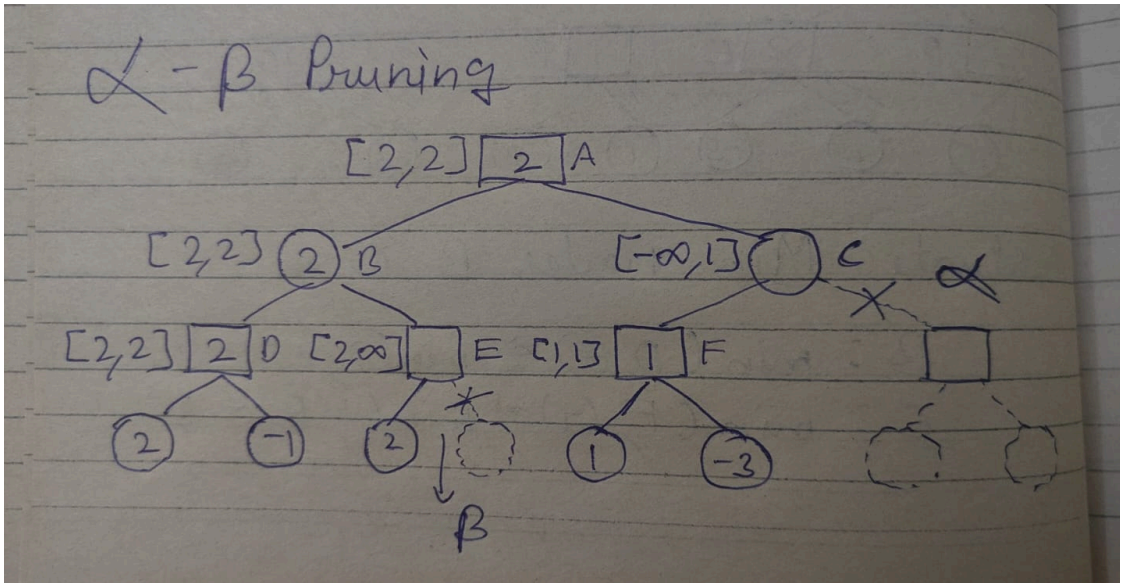
a) PART A

Minimax Search





Alpha Beta Search



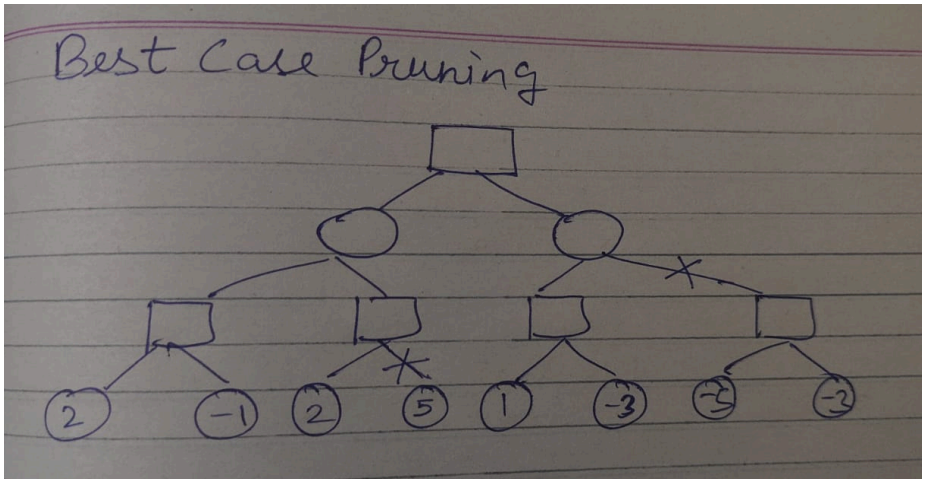
C is **alpha pruned**
E is **beta pruned**

Alpha Pruning: Search can be stopped below any MIN node having a beta value less than or equal to the alpha value of any of its MAX ancestors.

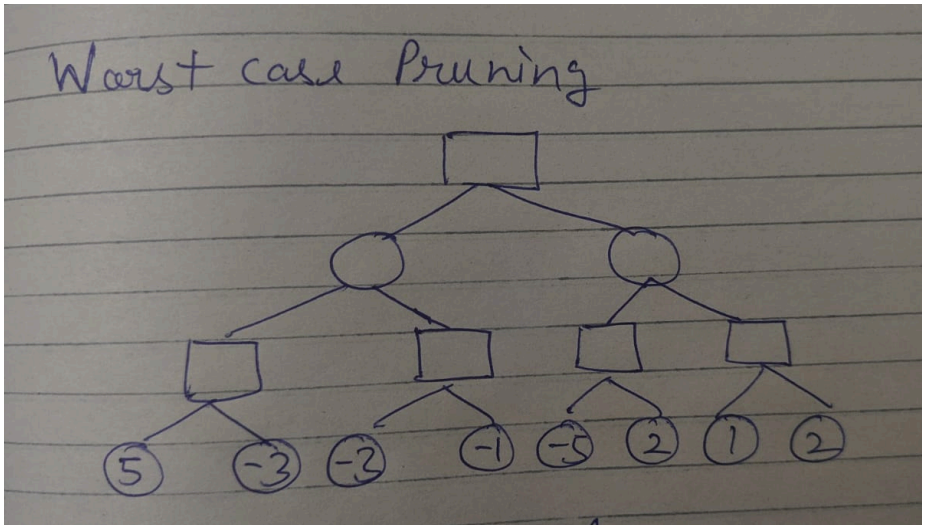
Beta Pruning: Search can be stopped below any MAX node having a alpha value greater than or equal to the beta value of any of its MIN ancestors

b) PART B

Best Case Rearrangement: we want to arrange the leaf nodes such that the leftmost branches contain the most favorable (extreme) values for both Max and Min nodes, leading to early alpha and beta updates and maximum pruning.



Worst Case Rearrangement: we want to arrange the nodes such that no pruning can occur. This will happen when the leftmost branches provide values that fail to trigger early updates to alpha or beta, forcing the algorithm to explore every single branch of the tree and search would run in Minimax fashion



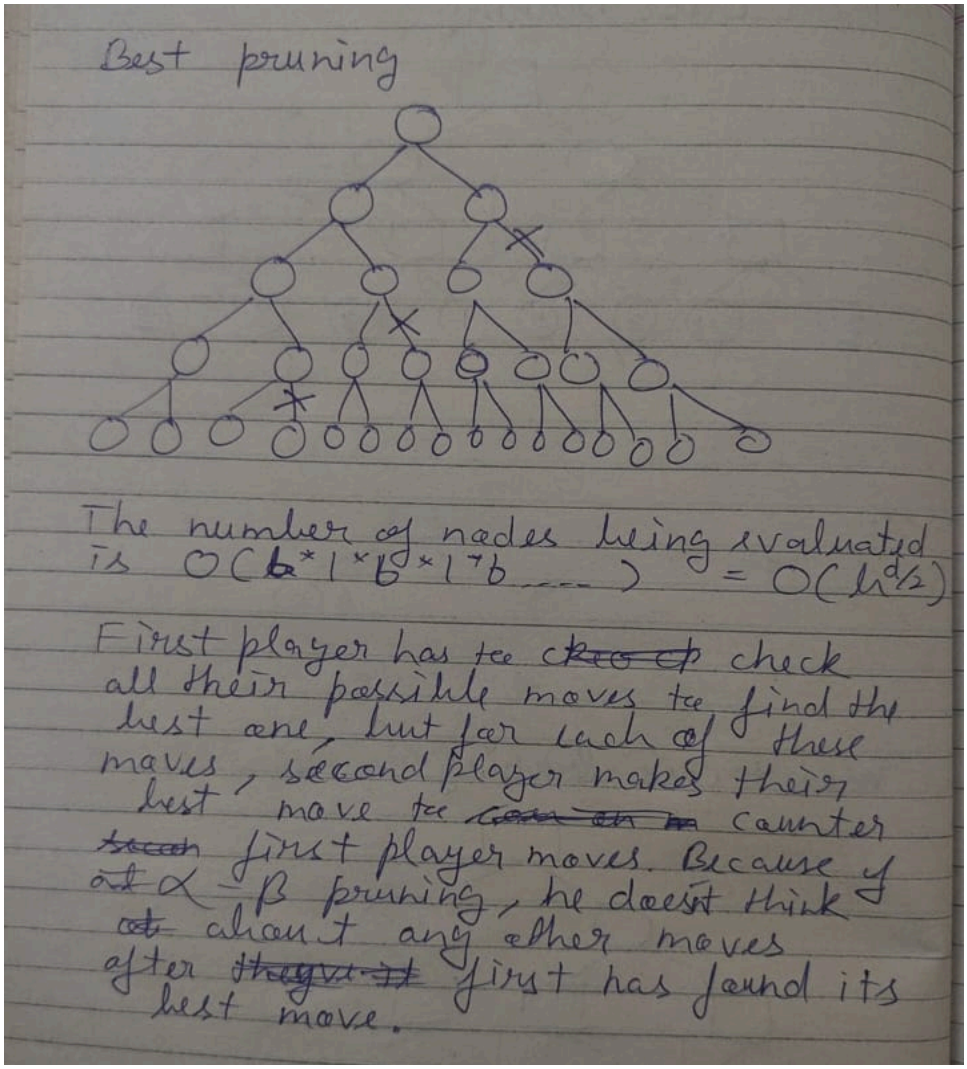
c) PART C

Branching Factor (b): Number of child nodes that each node can have.
Look-Ahead Depth (d): Depth of the tree, or how many levels deep we evaluate to make the best decision
The number of game states is exponential in the depth of the tree.
O(b*b*....) = O(b^d)

No algorithm can completely eliminate the exponent, but we can sometimes cut it in half, computing the correct minimax decision without examining every state by pruning large parts of the tree that make no difference to the outcome.

Pruning makes the search more efficient by ignoring portions of the search tree that make no difference to the optimal move.

Best Case Rearrangement: we want to arrange the leaf nodes such that the leftmost branches contain the most favorable (extreme) values for both Max and Min nodes, leading to early alpha and beta updates and maximum pruning.



Q3)

b)

```
Enter the start node: 1
Enter the end node: 2
Iterative Deepening Search Path: [1, 7, 6, 2]
Bidirectional Search Path: [1, 7, 6, 2]
A* Path: [1, 27, 9, 2]
Bidirectional Heuristic Search Path: [1, 27, 6, 2]
PS F:\AI\Search_2022201> python  "f:\AI\Search_2022201\code_2022201.py"
Enter the start node: 5
Enter the end node: 12
Iterative Deepening Search Path: [5, 97, 98, 12]
Bidirectional Search Path: [5, 97, 98, 12]
A* Path: [5, 97, 28, 10, 12]
Bidirectional Heuristic Search Path: [5, 97, 98, 12]
PS F:\AI\Search_2022201> python  "f:\AI\Search_2022201\code_2022201.py"
Enter the start node: 12
Enter the end node: 49
Iterative Deepening Search Path: None
Bidirectional Search Path: None
A* Path: None
Bidirectional Heuristic Search Path: None
PS F:\AI\Search_2022201> python  "f:\AI\Search_2022201\code_2022201.py"
Enter the start node: 4
Enter the end node: 12
Iterative Deepening Search Path: [4, 6, 2, 9, 8, 5, 97, 98, 12]
Bidirectional Search Path: [4, 6, 2, 9, 8, 5, 97, 98, 12]
A* Path: [4, 6, 27, 9, 8, 5, 97, 28, 10, 12]
Bidirectional Heuristic Search Path: [4, 34, 33, 11, 32, 31, 3, 5, 97, 28, 10, 12]
PS F:\AI\Search_2022201>
```

Uninformed Searches:

Paths for Iterative Deepening Search and Bidirectional Search Path are same in Public test cases; they might not always be identical because In IDS, the path depends on which nodes the algorithm happens to explore first. It might take a longer, less efficient path because it explores deeply. Where as In Bidirectional BFS, the path is more direct because the two searches meet in the middle

e) Informed Searches:

Path for A* Search and Bidirectional A* Search are different because In Bidirectional A* The search paths meet at some intermediate node, and the path is built from combining the forward and backward searches. This intermediate node might not lie on the optimal path that A* would have explored in a unidirectional search, leading to a different (though still valid) path.

c)

Uninformed Searches:

```
-----iterative_deepning_search-----
Total Time: 4144.9675
Total Space: 351223.3545
Total Cost: 14738102.9510
-----bidirectional_bfs-----
Total Time: 135.1965
Total Space: 166898.2188
Total Cost: 14899451.5280
```

e) Informed Searches:

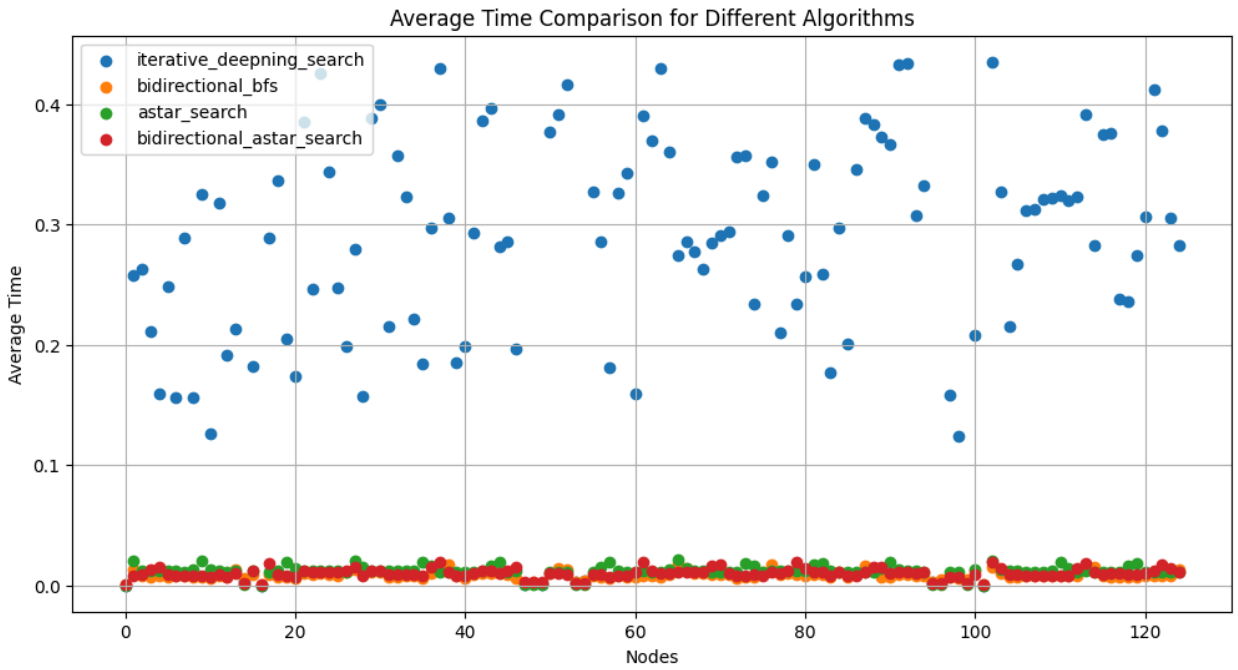
```
-----astar_search-----
Total Time: 187.0906
Total Space: 200127.1787
Total Cost: 13541530.7050
-----bidirectional_astar_search-----
Total Time: 158.2152
Total Space: 174604.1377
Total Cost: 13570546.3940
```

f)

How metrics are obtained : used initial all 125 nodes and calculated time, space and path cost of each search algorithm for all nodes u -> v finally for each node u in (0 to 124) I averaged out all the metrics for search algorithm to all v (0 to 124)

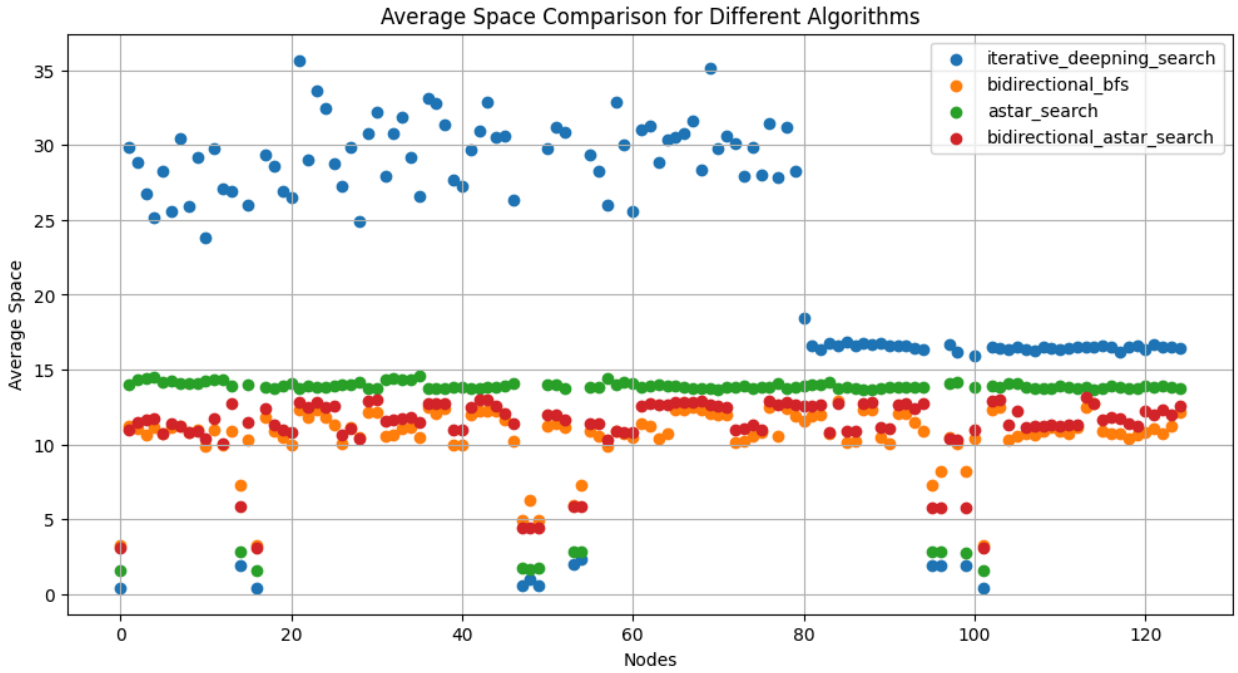
An uninformed search algorithm is given no clue about how close a state is to the goal(s).

Informed search strategy—one that uses Informed search domain-specific hints about the location of goals



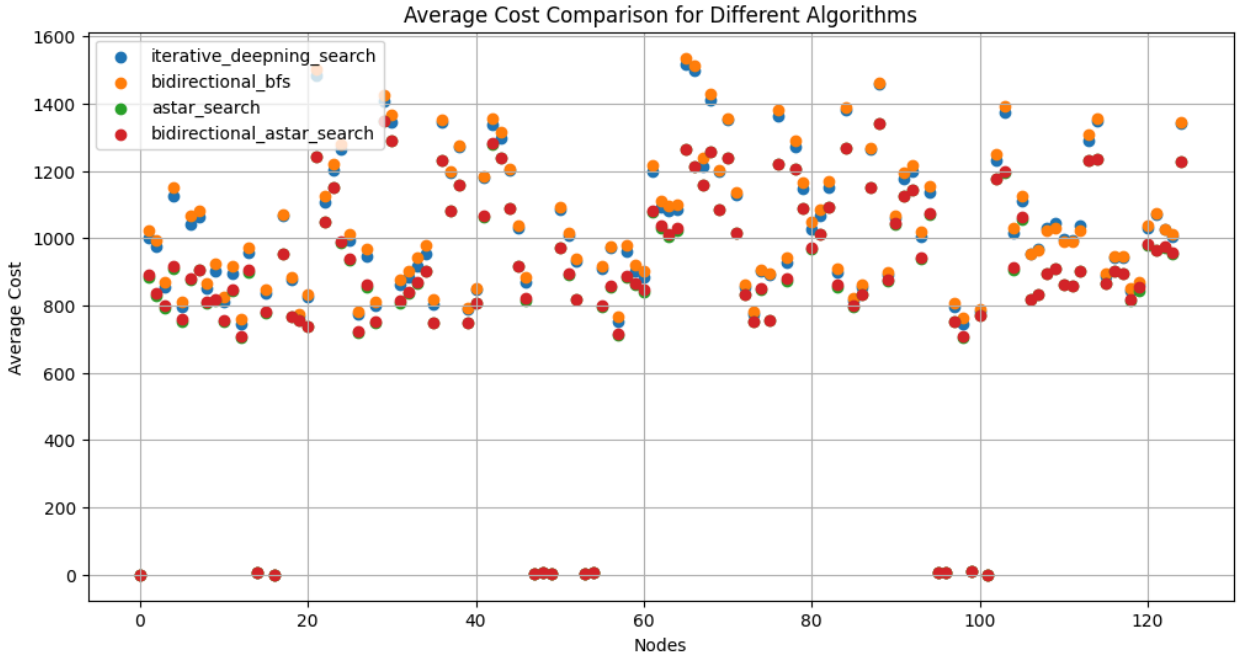
Time Graph shows that IDS is very slow compared to other search algorithms

IDS >> A* >= Bidirectional A* > Bidirectional BFS



Space(KB) complexity in order

IDS > A* > Bidirectional A* > Bidirectional BFS



In terms of cost of traveling or optimality we can observe that

Bidirectional BFS >= IDS > A* >= Bidirectional A*

Informed Searches can find solutions more efficiently than an uninformed strategy

g)

```
Bonus Problem: [(0, 49), (12, 57), (14, 53), (14, 99), (15, 35), (15, 46), (17,
45), (19, 100), (29, 42), (30, 42), (36, 38), (36, 114), (39, 40), (41, 70), (42
, 113), (43, 113), (47, 48), (47, 49), (50, 51), (53, 54), (53, 95), (55, 56), (
69, 124), (72, 73), (75, 106), (84, 114), (87, 88), (89, 90), (95, 96), (106, 10
7), (106, 111), (108, 109), (108, 111), (108, 112), (110, 111)]
```